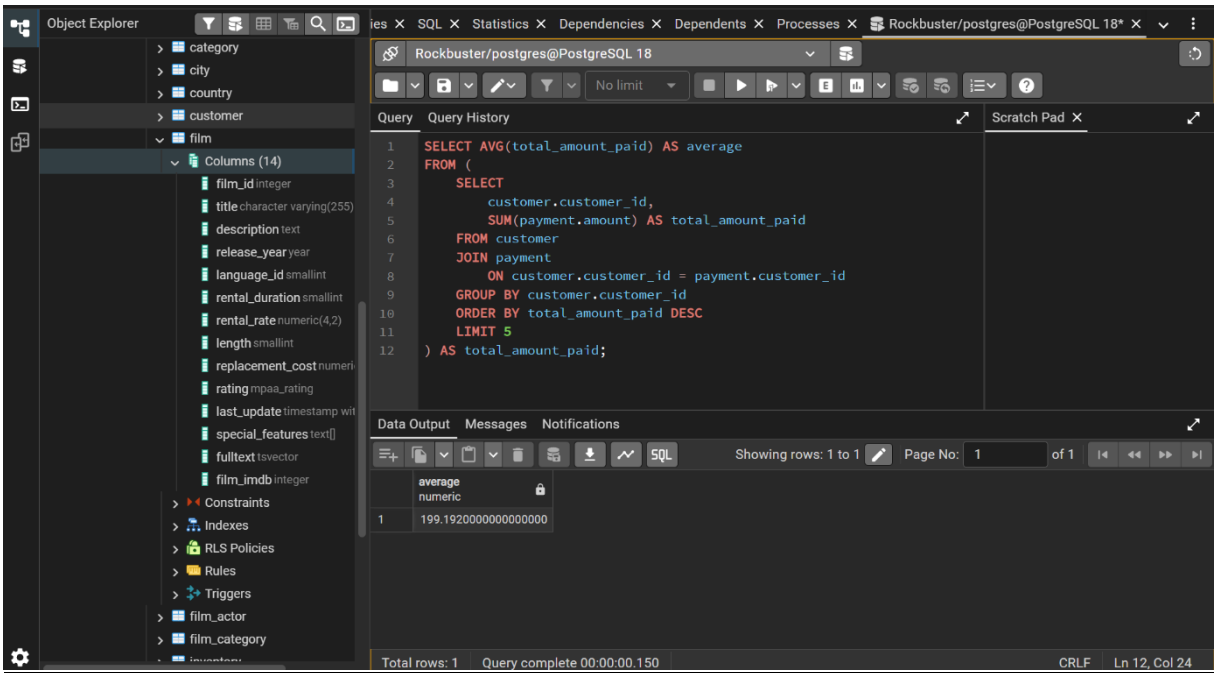
Performing Subqueries

Step 1: Find the average amount paid by the top 5 customers.

1. Copy the query you wrote in step 3 of the task from Exercise 3.7: Joining Tables of Data into the Query Tool. This will be your subquery, so give it an alias, "total_amount_paid," and add parentheses around it.
2. Write an outer statement to calculate the average amount paid.
3. Add your subquery to the outer statement. It will go in either the SELECT, WHERE, or FROM clause. (Hint: When referring to the subquery in your outer statement, make sure to use the subquery's alias, "total_amount_paid".)
4. If you've done everything correctly, pgAdmin 4 will require you to add an alias after the subquery. Go ahead and call it "average".



Query Applied:

```
SELECT AVG(total_amount_paid) AS average
FROM (
  SELECT
    customer.customer_id,
    SUM(payment.amount) AS total_amount_paid
```

```
  FROM customer
  JOIN payment
    ON customer.customer_id = payment.customer_id
  GROUP BY customer.customer_id
  ORDER BY total_amount_paid DESC
  LIMIT 5
) AS total_amount_paid;
```

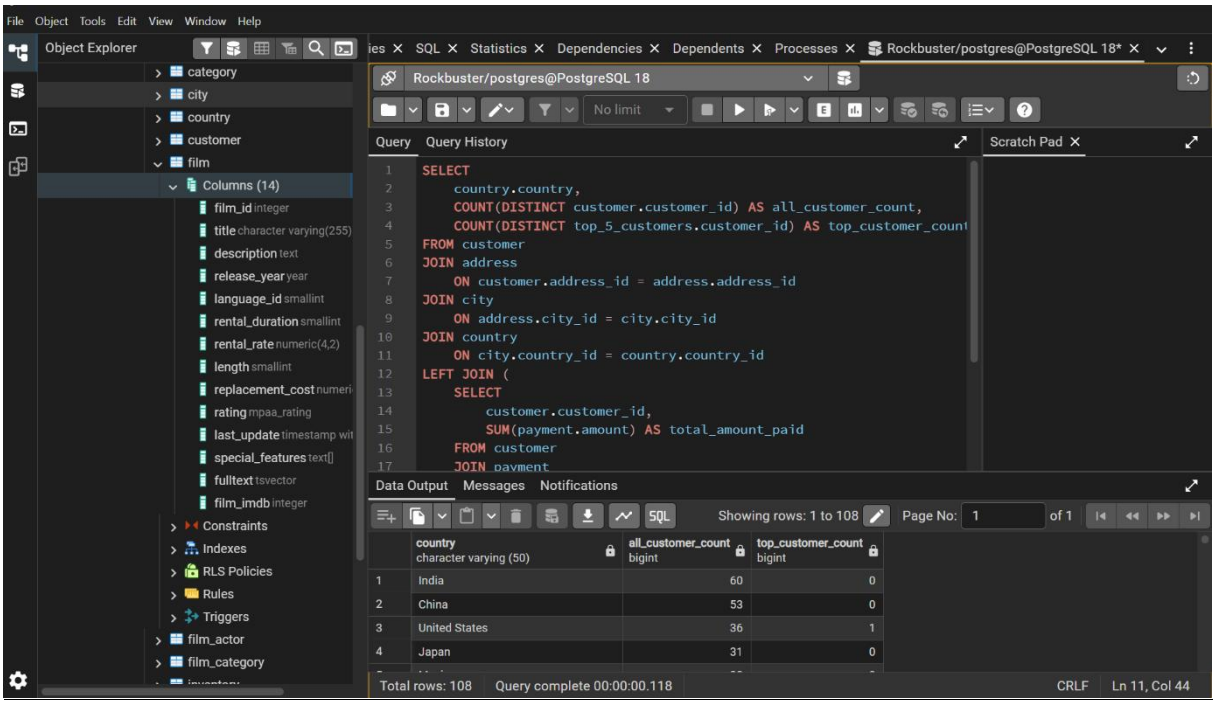
Answers:

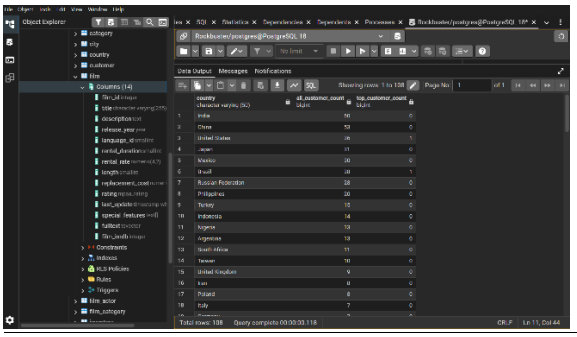
To compute the average payment of the top customers, I used my previous query that calculates total spending per customer. I enclosed this query in parentheses to convert it into a subquery and assigned it the alias total_amount_paid, as required. The outer query then applies the AVG() function to the derived column total_amount_paid. PostgreSQL requires an alias for subqueries in the FROM clause, so I used the alias average for the final output. This two-step structure separates ranking logic from aggregation logic. The Average is 199.1920000000000000

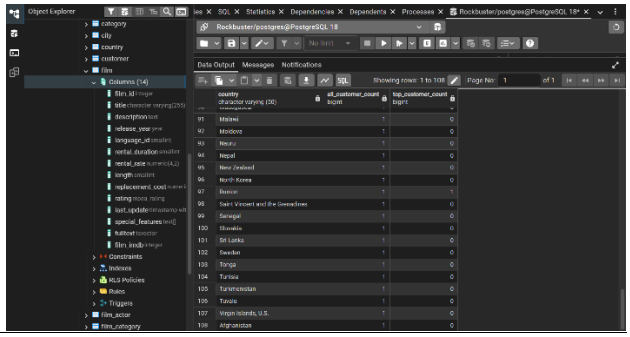
Step 2: Find out how many of the top 5 customers you identified in step 1 are based within each country.

Your final output should include 3 columns:

- "country"
- "all_customer_count" with the total number of customers in each country
- "top_customer_count" showing how many of the top 5 customers live in each country







Query Applied:

```
SELECT
    country.country,
    COUNT(DISTINCT customer.customer_id) AS all_customer_count,
    COUNT(DISTINCT top_5_customers.customer_id) AS top_customer_count
FROM customer
JOIN address
    ON customer.address_id = address.address_id
JOIN city
    ON address.city_id = city.city_id
JOIN country
    ON city.country_id = country.country_id
```

```
LEFT JOIN (
  SELECT
    customer.customer_id,
    SUM(payment.amount) AS total_amount_paid
  FROM customer
  JOIN payment
    ON customer.customer_id = payment.customer_id
  GROUP BY customer.customer_id
  ORDER BY total_amount_paid DESC
  LIMIT 5
) AS top_5_customers
ON customer.customer_id = top_5_customers.customer_id

GROUP BY country.country
ORDER BY all_customer_count DESC;
```

Answers:

To solve this problem, I first calculated the total number of customers per country by joining the customer table with address, city, and country tables. Since country information is not stored directly in the customer table, the joins follow the database relationships. Next, I reused the Top 5 Customers query as a subquery and left joined it to the main dataset. A LEFT JOIN ensures that all countries remain visible, even if none of the top customers belong to them. I then used COUNT(DISTINCT) to avoid duplicate counts and grouped the results by country. This allowed me to compare overall customer distribution with high-value customer concentration.

Relationship path:

customer → address → city → country

Explanation:

For Step 1:

I first calculated total spending per customer, ranked them, and limited the results to the top five. I then treated those results as a subquery and applied an AVG function to compute their average spending.

For Step 2:

I built a customer-country dataset using joins, then left joined my Top 5 Customers subquery to measure how high-value customers are distributed geographically. COUNT DISTINCT prevented duplication errors.

Write 1 to 2 short paragraphs on the following:

- Do you think steps 1 and 2 could be done without using subqueries?
- When do you think subqueries are useful?

Answers:

- Yes, steps 1 and 2 could technically be completed without subqueries, but the queries would likely become more complex and harder to read. For example, instead of isolating the top five customers first, I could attempt to combine all calculations into one large query using joins and aggregations. However, this would make the logic less clear and more difficult to debug. Subqueries help break a problem into smaller, logical parts, which makes the overall query easier to understand and maintain.
- Subqueries are especially useful when a calculation depends on the result of another query. They allow you to create temporary result sets that can be reused in outer queries, which improves clarity and structure. I find them helpful when ranking, filtering, or aggregating data in stages — particularly in analytical tasks like identifying top customers or comparing subsets of data. They also reduce repetition and make complex business questions much easier to solve.